

Exqutor 기반 캡스톤 연구 방향 설계안

작성일: 2026-03-28 (최종 확정: 2026-03-28)

목적: 팀 토의 → 연구제안서(4/2 마감) 초안의 근거 자료

버전: v2 — Phase 1~4 피드백 반영 확정판

배경: Exqutor 논문의 기초를 유지하면서, 논문이 다루지 않은 영역을 확장 실험하는 방향

핵심 아이디어 출처: 강재현 (3/28 팀 토의)

I. 출발점 — Exqutor가 해결한 것과 남겨놓은 것

Exqutor(arXiv:2512.09695v2)는 벡터 증강 분석 쿼리(VAQ)에서 카디널리티 오추정 문제를 해결했다. pgvector는 벡터 조건의 선택도를 33.3%로 고정 추정하고, VBASE는 50%, DuckDB는 100%로 추정하는데, 실제 선택도는 0.001%~90%까지 다양하므로 옵티마이저가 잘못된 실행 계획을 선택하게 된다. Exqutor는 두 가지 해법을 제시했다.

- **ECQO**: HNSW 인덱스가 있을 때, range query를 실행해 1~2ms 안에 정확한 카디널리티를 획득
- **Adaptive Sampling**: 인덱스가 없을 때, 모멘텀 기반 동적 샘플링으로 카디널리티를 근사

그 결과 pgvector에서 최대 1,000배, VBASE에서 10,000배, DuckDB에서 1.5~37배 성능 향상을 달성했다.

논문의 실험 조건 (고정된 변수들)

변수	논문이 사용한 값	현실과의 괴리
쿼리 유형	range query (<code>WHERE distance < D</code>)	실무 RAG는 거의 전부 top-k
필터 조건	없음 또는 태그 1개	실무는 다중 조건 복합 필터
벡터 조건	단일 벡터 유사도 조건 1개	벡터 조건 분해/다단계 적용 미탐구
HNSW 파라미터	M=16, ef_c=200, ef_s=400 고정	실무는 메모리/성능 트레이드오프로 다양
벡터 차원	96~128d (DEEP 96d, SIFT 128d)	OpenAI 1536d, Cohere 4096d
거리 함수	L2(유클리드)만	코사인 유사도가 더 흔함
pgvector 버전	0.7.x 시점	0.8.0에서 iterative scan 등 개선
병목 분석	실행 시간 위주	플래닝 시간 vs 실행 시간 분리 분석 없음

논문이 직접 인정한 한계 (Section 6.8)

1. 고차원 공간에서 샘플링 오버헤드 증가 — 정량화하지 않음
2. 기존 비용 모델의 부정확성 — 정확한 카디널리티를 줘도 비용 모델이 ANN 인덱스 특성을 반영 못함
3. 벡터 인덱스 맞춤 비용 모델 부재 — 향후 과제로만 언급

논문이 다루지 않은 근본 질문

Exqutor는 "카디널리티를 정확하게 추정하면 실행 계획이 좋아진다"까지 보여줬다. 하지만 VAQ의 전체 지연(latency) 중에서 플래닝 단계가 차지하는 비중이 얼마인지, 그리고 벡터 조건의 카디널리티 불확실성이 플래닝 시간 자체를 얼마나 증가시키지는 분석하지 않았다. 만약 플래닝이 실제 병목이라면, 카디널리티를 정확히 추정하는 것 외에도 벡터 조건의 구조를 바꿔서 플래닝 자체를 단순화하는 접근이 가능할 수 있다.

II. 핵심 연구 질문

"벡터 유사도 조건을 서로 다른 임계값의 두 단계로 분해(Cascaded Vector Similarity Decomposition)하면, (1) 쿼리 플래닝 시간을 단축할 수 있는가, (2) 중간 결과 캐싱으로 두 번 검색의 오버헤드를 상쇄할 수 있는가?"

이 질문은 강재현의 아이디어에서 출발한다. 재현의 핵심 관찰은 다음과 같다:

1. 관계형 필터와 벡터 검색이 결합된 쿼리에서, 플래닝이 오래 걸리는 이유는 **벡터 검색의 카디널리티를 모르기 때문이다**.
2. 만약 벡터 유사도 조건을 **느슨한 임계값(많이 retrieval)**과 **엄격한 임계값(적게 retrieval)** 두 단계로 나누면, 두 단계 사이의 카디널리티 차이는 명확하다.
3. 이 명확한 카디널리티 관계를 이용하면 **플래닝이 쉬워질 수 있다**.
4. 나아가 첫 번째 검색(느슨한 범위)의 결과 인덱스를 미리 잡아두면, 두 번째 검색(엄격한 범위)은 그 부분집합에서만 수행하므로 **두 번 검색의 오버헤드도 줄일 수 있다**.

학술적 맥락에서의 위치

이 아이디어는 데이터베이스 분야의 **Cascaded Filtering** 또는 **Coarse-to-Fine Search** 패러다임과 맞닿아 있다. 관계형 DB에서 조인 순서 최적화(Join Ordering)는 수십 년 연구된 고전 문제이고, ANN 검색에서도 coarse quantizer → fine re-ranking 같은 다단계 파이프라인이 표준이다(FAISS의 IVF+PQ 등). 그러나 **벡터 유사도 조건 자체를 쿼리 수준에서 분해하여 플래닝 복잡도를 줄이는 접근**은 기존 연구에서 다루지지 않았다.

핵심 가설: Exqutor가 "추정의 정확도"를 해결했다면, 우리는 "추정의 필요성 자체를 줄이는 구조적 접근"을 탐구한다.

하위 연구 질문 (3개 — RQ-단계 1:1 대응)

Phase 1에서 기존 RQ4(Exqutor 비교)를 RQ3에 흡수하여 3개로 압축. 캡스톤 규모에 적합하며, 각 RQ가 실험 단계와 1:1 대응.

RQ1 (병목 프로파일링 → 1단계): VAQ에서 전체 지연 중 플래닝 시간이 차지하는 비중은 얼마인가? 벡터 조건의 선택도에 따라 이 비중은 어떻게 변하는가? - *가설 H1:* 벡터 선택도가 낮을수록(< 1%) Planning Time 비율이 높아진다 (≥ 전체 latency의 20%). - *맞을 때:* Cascade 분해의 전제 조건 성립 → RQ2 진행 - *틀릴 때:* 플래닝은 병목이 아님 → "구조적 pre-filtering으로 실행 시간 최적화"로 재프레이밍 (IX장 대안 경로)

RQ2 (분해의 효과 → 2단계): 단일 벡터 유사도 조건을 두 단계(느슨→엄격)로 분해하면, 쿼리 성능(end-to-end latency)이 개선되는가? - *가설 H2:* Cascade 분해 시 end-to-end latency가 단일 조건 대비 개선된다. Materialized CTE의 실체화 오버헤드가 분해의 이점을 상쇄하지 않는다. - *맞을 때:* 핵심 기여 확보 → RQ3에서 ECQO 상호작용 분석 - *틀릴 때:* CTE 오버헤드가 이점을 상쇄 → Temp Table + Index 전략으로 전환, 또는 분해 임계값 간격 재조정

RQ3 (ECQO 상호작용 → 3단계): Cascade와 Exqutor ECQO는 보완적인가, 대체적인가? ECQO가 정확한 카디널리티를 제공한 상태에서도 Cascade 분해가 추가적 성능 향상을 가져오는가? - *가설 H3:* ECQO + Cascade 결합이 ECQO only보다 end-to-end latency에서 추가 향상을 보이며, 두 접근은 상호보완적이다. - *맞을 때:* Cascade + ECQO 병용 시너지 확인 → 실용적 기여 - *틀릴 때:* 총 비용 악화 → Cascade는 특정 조건에서만 유효, 조건 범위를 결론으로 제시

III. 연구 방향 — 3단계 구조

1단계: 병목 프로파일링 — "VAQ에서 뭐가 느린가"

목적: 재현 아이디어의 전제 조건 검증. "플래닝이 병목이라면 의미가 있다"는 가설을 먼저 확인.

실험 내용:

(a) 플래닝 vs 실행 시간 분리 측정

```
-- EXPLAIN (ANALYZE, TIMING, BUFFERS) 로 플래닝/실행 분리
EXPLAIN (ANALYZE, TIMING, BUFFERS, FORMAT JSON)
SELECT ... FROM products p
JOIN orders o ON p.id = o.product_id
WHERE p.embedding <-> :query < :D
      AND p.category = 'electronics'
      AND p.price BETWEEN 100 AND 500;
```

PostgreSQL의 EXPLAIN ANALYZE 출력에서: - **Planning Time**: 옵티마이저가 실행 계획을 수립하는 데 걸린 시간 - **Execution Time**: 실제 쿼리 실행 시간 - 이 둘의 비율을 선택도별로 측정

(b) 선택도 sweep

거리 임계값 D를 조절하여 벡터 선택도를 0.01% → 0.1% → 1% → 5% → 10% → 33% → 50% → 90%로 sweep. 각 지점에서: - Planning Time / Execution Time 비율 - pgvector 추정 행 수 vs 실제 행 수 → Q-error 계산 - 선택된 실행 계획 (Index Scan / Seq Scan / 조인 방식) - 필터 조건 수(0개, 1개, 2개, 3개)에 따른 변화

(c) 필터 복잡도별 플래닝 시간 변화

관계형 필터를 단계적으로 추가하면서 플래닝 시간이 얼마나 증가하는지 측정:

```
-- Level 0: 벡터 조건만
WHERE embedding <-> :query < :D

-- Level 1: + 관계형 필터 1개
WHERE embedding <-> :query < :D AND category = 'electronics'

-- Level 2: + 관계형 필터 2개
WHERE embedding <-> :query < :D AND category = 'electronics' AND price BETWEEN 100 AND 500

-- Level 3: + 조인
FROM products p JOIN orders o ON ... WHERE ...
```

핵심 산출물: - **Planning Time vs Execution Time 비율 곡선** — 선택도별, 필터 수별 - **선택도 vs Q-error 곡선** — "33.3% 고정 추정이 우연히 맞는 교차점" 시각화 - **"플래닝 시간이 병목인 영역" 식별** — 재현 아이디어가 유효한 조건 범위 특정

판단 기준: 만약 Planning Time이 전체 latency의 10% 미만이라면, 플래닝 단축보다 실행 시간 단축이 더 중요하므로 2단계 실험의 초점을 조정한다. 반대로 Planning Time이 유의미하면(20%+), 재현 아이디어의 효용성이 높아진다.

사전 예측 (솔직한 리스크 평가): PostgreSQL의 Planning Time은 일반적으로 Execution Time의 1% 미만인 경우가 많다. 특히 단순 VAQ(테이블 1~2개, 필터 1~2개)에서는 거의 항상 미미하다. 따라서 **1단계에서 "플래닝 병목 아님"이 나올 가능성이 높다**. 이 경우 2단계의 프레이밍을 "플래닝 단축"에서 **"Cascade 구조 자체가 옵티마이저에게 더 나은 힌트를 주어 실행 계획 품질을 개선하는 효과"**로 전환한다. 구체적으로:

- **전환 시나리오 A** (Planning Time < 10%): Cascade의 효과를 "Stage 10 pre-filter 역할을 하여 Stage 2의 실행 시간을 단축하는 구조적 이점"으로 재프레이밍. 이 경우 end-to-end latency(Planning + Execution) 비교가 핵심 지표가 됨
- **전환 시나리오 B** (Planning Time < 10% + Cascade 이점 없음): 패키지 2(비용 모델 검증)로 완전 전환. B-1 선택도 sweep → A-1 비용 모델 실증 → C-1 복합 필터
- **긍정 시나리오** (Planning Time ≥ 20%): 원래 계획대로 진행

논문과의 차별점: Exqutor 논문은 Planning Time과 Execution Time을 분리 분석하지 않았다. 플래닝 오버헤드 자체가 문제인지 아닌지에 대한 정량적 데이터가 없다.

참고 문헌 근거 (번호는 Exqutor 논문 arXiv:2512.09695v2의 참고문헌 목록 기준): - [54] Datta et al.: 카디널리티 추정 오류가 실행 계획 선택에 미치는 영향 정량화 - [76] ACORN: 선택도에 따른 최적 전략 전환 확인 - [77] SERF: 선택도 오추정 ±30%에서 1.3배, ±50%에서 2배 성능 저하 관측

2단계: 핵심 실험 — Cascaded Vector Similarity Decomposition

배경 (재현 아이디어 정식화):

코사인 유사도의 범위는 $[-1, 1]$ 이다. 유사도가 높을수록(1에 가까울수록) 검색 결과가 적어지고(카디널리티↓), 유사도가 낮을수록 많아진다(카디널리티↑). 재현의 아이디어는 이 성질을 이용하는 것이다.

기존 방식 (단일 검색):

```
-- 한 번에 엄격한 조건으로 검색 (L2 거리 기준, pgvector <-> 연산자)
SELECT * FROM items
WHERE embedding <-> :query < 0.5 -- 카디널리티 불확실
      AND category = 'electronics'
      AND price BETWEEN 100 AND 500;
```

→ 옵티마이저는 `<-> :query < 0.5` 의 결과가 몇 건인지 모르므로 플래닝이 어려움.

제안 방식 (Cascaded Decomposition)

```
-- Stage 1: 느슨한 임계값으로 먼저 걸러내기 (L2 거리 기준)
WITH MATERIALIZED coarse_set AS (
  SELECT *, embedding <-> :query AS dist
  FROM items
  WHERE embedding <-> :query < 2.0 -- 넓은 범위, 카디널리티 높음 (예측 쉬움)
)
-- Stage 2: 좁은 범위에서 정밀 검색
SELECT * FROM coarse_set
WHERE dist < 0.5 -- 좁은 범위, coarse_set의 부분집합
      AND category = 'electronics'
      AND price BETWEEN 100 AND 500;
```

참고: pgvector에서 L2 거리는 `<->` 연산자, 코사인 거리는 `<=>` 연산자, 내적은 `<#>` 연산자를 사용한다. `cosine_similarity()` 같은 함수는 제공되지 않는다. L2 거리는 값이 작을수록 유사하고($< D$), 코사인 거리는 `1 - cosine_similarity` 이므로 역시 값이 작을수록 유사하다. 본 설계안의 모든 예시는 **L2 거리 기준**으로 통일한다. 3단계 확장(3-E)에서 코사인/내적으로 전환 실험을 수행한다.

왜 플래닝이 쉬워질 수 있는가: - Stage 1(`dist < 2.0`)의 카디널리티는 상대적으로 높고 예측 가능 (전체의 큰 비율) - Stage 2는 Stage 1 결과의 부분집합이므로 범위가 명확 - 두 단계의 카디널리티 관계가 $\text{card}(\text{Stage2}) \leq \text{card}(\text{Stage1})$ 로 보장되므로, 옵티마이저가 결정해야 할 불확실성이 줄어듦

왜 두 번 검색해도 느려지지 않을 수 있는가 (재현의 두 번째 관찰): - Stage 1 결과의 row ID/인덱스를 메모리에 캐싱 - Stage 2는 캐싱된 결과 집합 내에서만 유사도 비교 → 전체 테이블 재스캔 불필요 - 실질적으로 Stage 1이 pre-filter 역할을 함

실험 설계:

(a) 분해 임계값 조합 테스트

Stage 1 임계값 (느슨)	Stage 2 임계값 (엄격)	기대 카디널리티 관계
<code>dist < 5.0</code>	<code>dist < 0.5</code>	$\text{Stage1} \gg \text{Stage2}$
<code>dist < 2.0</code>	<code>dist < 0.5</code>	$\text{Stage1} > \text{Stage2}$
<code>dist < 1.0</code>	<code>dist < 0.5</code>	$\text{Stage1} \approx \text{수배} \times \text{Stage2}$
<code>dist < 0.7</code>	<code>dist < 0.5</code>	$\text{Stage1} \approx \text{Stage2}$ (분해 무의미)

임계값 결정 방법: 실제 D 값은 데이터셋에 따라 다르므로, 1단계 선택도 sweep 결과를 참조하여 결정한다. 위 수치는 SIFT1M(128d, L2) 기준 예시이며, 실제로는 선택도(0.01%, 1%, 10%, 50%)에 대응하는 D 값을 먼저 측정한 뒤 사용한다.

각 조합에서: - 단일 검색 vs 분해 검색의 Planning Time 비교 - 단일 검색 vs 분해 검색의 Execution Time 비교
- **Total Time = Planning + Execution** 비교 (이것이 최종 판단 기준)

(b) 관계형 필터 복합도에 따른 효과 변화

필터 없음 (벡터만) → 필터 1개 → 필터 2개 → 필터 3개 + 조인

복합 필터가 많을수록 플래닝 공간이 커지므로, 분해의 효과가 더 클 수 있다는 가설 검증.

(c) Exqutor ECQO와의 비교 및 조합

전략	설명
Baseline	pgvector 기본 (33.3% 고정 추정)
ECQO only	Exqutor의 정확한 카디널리티 추정
Cascade only	벡터 조건 분해 (ECQO 없이)
ECQO + Cascade	정확한 카디널리티 + 벡터 조건 분해

핵심 질문: ECQO와 Cascade가 **같은 문제를 다른 방식으로 해결하는가**(대체적), 아니면 **서로 다른 문제를 각각 해결하는가**(보완적)?

- ECQO: "카디널리티를 정확히 알려주자" → 옵티마이저가 올바른 계획 선택
- Cascade: "카디널리티를 몰라도 되는 구조로 바꾸자" → 플래닝 자체를 단순화
- 만약 보완적이라면 두 기법을 결합하면 추가 향상이 가능할 것

(d) 중간 결과 캐싱/인덱싱 전략

재현의 아이디어 중 "첫 번째 검색 결과의 인덱스를 미리 잡아두면 두 번째 검색이 빨라질 수 있다"는 부분 검증:

- **Strategy A:** CTE (Common Table Expression) — PostgreSQL이 자동 처리
- **Strategy B:** Materialized CTE — 중간 결과를 임시 테이블로 실체화
- **Strategy C:** Temporary table + Index — 중간 결과에 인덱스까지 생성

각 전략의 오버헤드와 Stage 2 가속 효과를 측정.

실험 매트릭스 축소 프로토콜:

전체 조합: 분해 조합 4 × 필터 수 4 × ECQO 유무 4 × 캐싱 전략 3 = 192개. 이를 2단계로 축소한다.

- **Phase 2a (1주, 12~16개 조합):** 핵심 축만 실험
- 분해 조합 4가지 × ECQO 유무 4전략 = 16개 조합
- 필터 없음(벡터만)으로 고정, 캐싱은 Materialized CTE만
- 판단: 분해 효과가 유의미한 조합 식별 (10% 이상 총 비용 차이)
- **Phase 2b (2주, 유의미한 조합만 확장):** Phase 2a에서 유의미한 분해 조합만 선택 → 필터 수 4단계 × 캐싱 전략 3가지로 확장
- 예상 조합: 2~3개 유의미한 분해 × 4 필터 × 3 캐싱 = 24~36개

실험 측정 프로토콜:

- **반복 측정:** 각 조합 최소 5회 실행, 첫 1회는 워밍업으로 제외, 나머지 4회의 중앙값 사용
- **워밍업:** 매 조합 시작 전 동일 쿼리 1회 실행 (shared_buffers 채우기)
- **아웃라이어 처리:** 4회 중 최대값 제거, 나머지 3회 평균
- **유의미한 차이 기준:** 총 비용(Planning + Execution)에서 10% 이상 차이
- **환경 통제:** `pg_stat_statements` 리셋, 다른 세션 없는 단독 실행
- **end-to-end latency 측정:** ECQO 오버헤드(1~2ms)와 Cascade 총 오버헤드(Stage1 + 실체화 + Stage2)를 동일 단위로 비교하여 공정한 비교 보장

CTE 기술 타당성 참고: PostgreSQL 12+에서 비재귀 CTE는 자동 인라인 가능(optimization fence 제거).

MATERIALIZED 키워드로 강제 실체화할 수 있지만, 실체화 자체의 오버헤드(임시 테이블 기록)가 플래닝 절감분을 상쇄할 수 있음. 이를 Phase 2a에서 먼저 확인한다.

핵심 산출물: - "분해 임계값 × 총 비용" 최적점 그래프 — 느슨-엄격 임계값 간격이 어느 정도일 때 총 비용이 최소화되는지 - "Planning Time 절감 vs Execution Time 증가" 트레이드오프 곡선 — 분해가 유리한 영역과 불리한 영역의 경계 - ECQO vs Cascade vs 결합의 성능 비교표 — 보완적/대체적 관계 판별 - 캐싱 전략별 효과 비교 — 중간 결과 처리의 최적 방법

논문과의 차별점: Exqutor 논문은 카디널리티 추정의 정확도를 높이는 것에 집중했다. 우리는 "벡터 조건의 구조를 바꿔서 플래닝 부담을 줄이는 접근"이라는 완전히 다른 차원의 최적화를 탐구한다. 이는 Exqutor를 부정하는 것이 아니라, Exqutor와 **직교하는 최적화 축**을 발견하는 것이다.

참고 문헌 근거 (번호는 Exqutor 논문 arXiv:2512.09695v2의 참고문헌 목록 기준): - [48] FAISS: IVF + PQ의 coarse-to-fine 파이프라인 — 인덱스 수준의 다단계 검색의 효과를 쿼리 수준으로 확장하는 아이디어 - [41] Filtered-DiskANN: 선택도별 검색 범위를 동적 조정 — 선택도에 따른 전략 분기의 중요성 - [74] Consistent Flexible Selectivity: $\text{selectivity}(A \text{ AND } B) = \text{selectivity}(A) \times P(B|A)$ — Cascade 구조에서 조건부 선택도 추정의 이론적 기반 - [56] Progressive Optimization: 쿼리 실행 중에 카디널리티를 점진적으로 개선 — 우리의 "단계적 검색"과 유사한 정신

3단계: 확장 실험 — 논문이 고정한 변수를 하나씩 바꾸기

여력에 따라 1~2개 선택. 모든 확장 실험은 2단계 위에 변수를 하나 추가하는 형태로 독립 수행 가능하다.

3-A. top-k로의 전환

질문: range query에서 확인한 Cascade 효과가 top-k(`ORDER BY distance LIMIT k`)에서도 유효한가?

방법: 2단계 동일 쿼리를 top-k(k=10/100/1000)로 변환, Cascade 효과 비교. k가 카디널리티를 제한하므로 효과 약화 가능하나 복잡 조인에서는 유의미할 것으로 예상.

3-B. HNSW 파라미터 변화

질문: M/ef_search 저하 시 ECQO range query가 부정확해지는가? Cascade가 더 강건한 대안인가?

방법: M=4/8/16/32, ef_search=40/100/200/400에서 ECQO 정확도와 Cascade 효과를 동시 측정. ef_search 감소 시 Cascade가 카디널리티 추정 불필요로 더 강건할 가능성.

3-C. 고차원 벡터 (1536d+)

질문: 논문이 인정한 한계 #1 — 고차원에서 ECQO 오버헤드가 얼마나 증가하는가? Cascade의 Stage 1 오버헤드도 함께 증가하는가?

방법: sentence-transformers(768d)/OpenAI(1536d) 임베딩으로 동일 실험 반복, 차원별 오버헤드 측정.

3-D. pgvector 0.8.0 비교

질문: pgvector 0.8.0의 자체 개선(iterative scan, 비용 추정 공식 수정)으로 Exqutor/Cascade의 필요성이 줄어들었는가?

방법: 동일 1~2단계 실험을 pgvector 0.8.0에서 반복, iterative scan 영향 분석.

3-E. 거리 함수 변경 (L2 → 코사인 → 내적)

질문: 거리 함수가 바뀌면 Cascade의 분해 임계값 설정과 효과가 달라지는가?

방법: L2/코사인/내적 각각 인덱스 빌드 후 2단계 실험 반복. 거리 함수별 최적 분해 임계값 탐색.

IV. 실험 환경

소프트웨어

구성 요소	버전/설정
PostgreSQL	16 (Exqutor 패치 적용 빌드)
pgvector	0.7.x (논문 재현용) + 0.8.0 (비교용)
pg_hint_plan	최신 (실행 계획 강제 지정용)
Python	3.10+ (벤치마크 스크립트, 결과 분석)
TPC-H	SF1 또는 SF10 (데이터 규모는 하드웨어에 따라 결정)

하드웨어 (미확정)

- 옵션 A: 연구실 서버 (교수님께 요청 필요)
- 옵션 B: 팀원 개인 머신 (소규모 SF1로 시작)
- 결정 필요: 세은이 교수님께 서버 사용 가능 여부 확인 중

데이터셋

- 기본: TPC-H VAQ 벤치마크 (Exqutor GitHub의 [Vector-augmented_SQL_analytics/](#) 스크립트 활용)
- 벡터 데이터: SIFT1M(128d) 기본, sentence-transformers(768d) 또는 OpenAI(1536d) 확장용
- 데이터 로딩: Exqutor GitHub의 [insert_data_pgvector.sh](#) 활용

Exqutor 코드 상태 (GitHub 확인 결과)

- 구조: PostgreSQL 16 소스에 패치 파일 적용 → 수정된 PostgreSQL 빌드
- 빌드 방법: [apply_patch.sh](#) → [build.sh](#)
- 벤치마크: TPC-H VAQ 8개 쿼리(Q3, Q5, Q8, Q9, Q10, Q11, Q12, Q20) + 데이터셋 다운로드 스크립트
- 마지막 커밋: 2025-11-12 (paper artifact 수준, 활발한 유지보수 없음)

V. 예상 일정

기간	마일스톤	산출물
~4/2	연구제안서 제출	연구제안서 PDF (4페이지)
4/1~4/14	환경 구축	PostgreSQL+pgvector+pg_hint_plan 설치, TPC-H 데이터 로딩, EXPLAIN 파싱 스크립트
4/14~4/28	1단계 실험 (병목 프로파일링)	Planning vs Execution 비율 곡선, Q-error 곡선, 병목 영역 식별
4/28~5/15	2단계 실험 (Cascaded Decomposition)	분해 임계값 최적점, ECQO와의 비교, 캐싱 전략 비교
5/15~5/25	3단계 확장 (택 1~2개)	확장 실험 결과
5월 말	중간발표 + 중간보고서	발표 자료, 보고서
6월	결과 정리 + 최종 보고서	최종발표, 최종보고서, 전시회

VI. 예상 산출물 요약

1. Planning Time vs Execution Time 비율 프로파일 — 1단계: VAQ 병목의 정량적 분석
2. 선택도 vs Q-error 곡선 + crossover point 맵 — 1단계: 기존 추정이 틀리는 영역 시각화
3. "분해 임계값 × 총 비용" 최적점 그래프 — 2단계 핵심: Cascade의 sweet spot 탐색
4. ECQO vs Cascade vs 결합 성능 비교표 — 2단계: 두 접근의 관계 판별
5. 중간 결과 캐싱 전략 효과 비교 — 2단계: 구현 수준의 가이드라인
6. 확장 실험 결과 (top-k, 파라미터, 고차원 등) — 3단계
7. 실무 가이드라인: "이런 조건에서는 Cascade가 유리하고, 저런 조건에서는 ECQO가 더 낫다" — 종합

VII. 스토리라인 (연구제안서 서사 구조)

벡터 증강 분석 쿼리(VAQ)는 벡터 유사도 검색과 관계형 분석을 결합하는 차세대 데이터 분석의 핵심이다. 기존 RDBMS의 쿼리 옵티마이저는 벡터 조건의 선택도를 정확히 추정하지 못하여 잘못된 실행 계획을 생성하고, Exqutor(2025)는 ECQO와 Adaptive Sampling으로 이 카디널리티 추정 문제를 해결했다.

그러나 VAQ의 성능 병목은 카디널리티 추정의 부정확성만이 아닐 수 있다. 벡터 조건이 포함된 복잡한 쿼리에서 **벡터 조건의 구조 자체를 바꿔 실행 효율을 개선할 여지**가 남아 있다. 벡터 유사도 조건을 느슨한 단계와 엄격한 단계로 분해하면, (1) 각 단계의 카디널리티 관계가 예측 가능해져 옵티마이저의 계획 수립이 용이해지고, (2) 느슨한 단계의 결과를 캐싱하여 엄격한 단계의 실행을 가속할 수 있다.

본 연구는 이 관점에서 **Cascaded Vector Similarity Decomposition**을 제안하고 실험적으로 검증한다. 먼저 VAQ의 성능 병목을 Planning Time과 Execution Time으로 분리하여 정량적으로 프로파일링하고, Cascade 분해가 end-to-end latency를 개선하는 조건을 체계적으로 탐색한다. 이 접근이 Exqutor의 ECQO와 상호보완적인지, 어떤 조건에서 각각 유리한지를 분석한다.

(대안 스토리라인 — 1단계에서 플래닝이 병목이 아닌 경우): 만약 플래닝이 병목이 아닌 것으로 확인되면, Cascade의 효과를 "구조적 pre-filtering을 통한 실행 시간 최적화"로 재프레이밍한다. 이 경우에도 "벡터 조건 분해가 총 비용을 줄이는가"라는 연구 질문은 유효하다.

VIII. 팀 작업 분담

4인 1팀으로 운영하되, 단계별로 역할을 유동적으로 배분한다.

환경 구축 (4/1~4/14, 전원): - PostgreSQL 16 + Exqutor 패치 빌드 (소스 컴파일) - pgvector, pg_hint_plan 설치 - TPC-H VAQ 데이터 생성 및 로딩 - EXPLAIN ANALYZE 결과 자동 수집 + Planning/Execution 분리 Python 스크립트 구축 - 벡터 데이터셋(SIFT1M) 로딩 및 인덱스 빌드

1~2단계 실험 (4/14~5/15, 전원): - 1단계: 병목 프로파일링 실험 실행 - 2단계: Cascade 쿼리 설계 및 분해 임계값 조합 테스트 - 중간 결과 캐싱 전략 구현 및 비교 - EXPLAIN JSON 파싱 및 시각화

3단계 확장 + 보고서 (5/15~6월, 전원): - 확장 실험 택 1~2개 수행 - 중간발표 자료 작성 - 최종보고서 작성

IX. 토의 포인트 (팀원 확인 필요)

1. [최우선] 병목이 정말 플래닝인지 먼저 확인 — 간단한 VAQ로 Planning Time vs Execution Time 비율을 측정해봐야 함. 결과에 따라 2단계 방향이 달라짐
2. pgvector에서 range query가 HNSW 인덱스를 타는지 확인 — `WHERE embedding <-> query < D` 형태가 인덱스를 쓰는지. 안 쓰면 ECQO 비교 실험 자체가 어려움
3. CTE 기반 Cascade가 PostgreSQL에서 실제로 플래닝을 분리하는지 확인 — PostgreSQL이 CTE를 인라인하면(optimization fence 제거) 분해 효과가 사라질 수 있음. `MATERIALIZED` 키워드로 강제 가능
4. Exqutor 코드 빌드 가능 여부 — 누가 먼저 clone해서 `apply_patch.sh && build.sh` 돌려볼 것인가
5. 연구실 서버 사용 가능 여부 — 세은이 교수님께 확인 중. 결과에 따라 데이터 규모(SF1 vs SF10) 결정
6. 3단계 확장 실험 우선순위 — top-k(3-A)가 가장 실무 관련성 높고, pgvector 0.8.0 비교(3-D)가 시의성 높음

X. 피드백 반영 이력

출처	피드백	반영 위치	상태
자문위원(박성원)	데이터셋·대조군·평가지표 구체화	II~IV장 + 실험 설계 전반	✅ 반영
박세은 (3/28)	중복 필터링 시나리오 부자연스러울 수 있음	IX장 토의 포인트 + VII장 대안 스토리라인	✅ 반영
강재현 (3/27)	플래닝 메커니즘 먼저 조사	1단계 병목 프로파일링으로 설계	✅ 반영
교수님 미팅 (3/28)	방향 확정 승인	전체	✅ 확정
환경구축 시 검증	range query HNSW 사용 여부(IX-#2), CTE 플래닝 분리(IX-#3)	IX장	⌚ 환경구축 시
외부 대기	Exqutor 빌드 검증(IX-#4), 연구실 서버(박세은 확인 중)	IX장	⌚ 외부

XI. 변경 이력

날짜	버전	변경 내용
2026-03-28	v1	초안 작성 (강재현 아이디어 기반, 팀 토의 반영)
2026-03-28	v2	RQ 4→3개 압축 (RQ4를 RQ3에 흡수), 가설 수치 기준 명시, 실패 시 대안 경로 정의, 피드백 반영 이력 추가